PCPOP: A Julia library for polynomial optimization with partially commutative variables

Moisés Bermejo Morán[2] and Abhishek Mishra[1]

[1]Laboratoire d'Information Quantique, Université libre de Bruxelles, Belgium
[2]Bilkent University, Türkiye

September 29, 2025

PCPOP implements mathematical structures and new algorithms to represent and manipulate elements of free algebra, partially-commutative algebra, graph product algebra and space of trace polynomials. The main objective is to leverage the equality constraints to reduce the complexity of the SDP relaxations for polynomial optimization problems. Given the set of equality constraints G with subset S of them being special binomial constraints such as partial commutations, projectors, unitaries, dichotomic, orthogonality etc., PCPOP finds normal forms directly and efficiently for S and then for the remaining equality constraints $G \setminus S$, it uses Macaulay matrix in the quotient space modulo S to reduce the polynomials. For trace polynomial optimization, it finds normal forms with respect to cyclic equivalence, partial commutations and other special binomial constraints.

1 Introduction

PCPOP is a Julia library that builds and solves SDP relaxations for polynomial optimization with partially-commutative variables, where some variables commute and some not, and hence generalizing the non-commutative and fully-commutative polynomial optimization. It supports both the eigenvalue and tracial versions.

Polynomial optimization has become a central tool in many diverse fields, but it is an NP-hard problem. Lasserre first developed a numerical algorithm to solve the fully-commutative polynomial optimization problems[Las01]. This was later extended to non-commutative variables, yielding the NPA hierarchy [PNA10; NPA08]. Given the natural non-commutativity of operators in quantum physics, the NPA hierarchy finds numerous applications in quantum foundations [Tav+24], device-independent quantum cryptography [Pri+23], many-body physics[Wan+24], and others. It has also been extended to trace polynomials [Bur+13], with many prominent applications, such as in quantum networks and quantum cryptography in prepare-and-measure scenarios [Tav+22b; Tav+22a].

Given $p(\underline{x}), q_i(\underline{x})$, polynomials in $\underline{\mathrm{x}}$=$(x_1, x_2 ... x_n)$ variables, the fully-commutative polynomial optimization reads as,

$$\begin{aligned} p^* &= \underset{\underline{x}}{inf}\, p(\underline{x}), \\ & s.t\, q_i(\underline{x}) \geq 0. \end{aligned} \tag{1}$$

In non-commutative variables X=$(x_1, x_2...x_n)$ and polynomials $p(X), q_i(X), r_i(X)$ in free algebra, the eigen-value version of non-commutative polynomial optimization reads as,

$$\begin{aligned} p^* = \inf_{\underline{A}\in\mathcal{B}(\mathcal{H})^n,\psi\in\mathcal{H},\mathcal{H}} & \langle\psi, p(\underline{A})\psi\rangle\,, \\ s.t\, q_i(\underline{A}) &\geq 0, \\ s.t\ \langle\psi, r_i(\underline{A})\psi\rangle &\geq 0, \\ s.t\ \langle\psi,\psi\rangle &= 1. \end{aligned} \tag{2}$$

where $p(\underline{A})$ means the evaluation of polynomial p at operators $\underline{A}$. Check [PNA10] for details.

In non-commutative variables X=$(x_1, x_2...x_n)$ and polynomials $p(X), q_i(X), r_j(X)$, in free algebra, the tracial version of non-commutative polynomial optimization reads as,

$$\begin{aligned} p^* = & \inf_{\underline{A}\in\mathcal{B}(\mathcal{H})^n,\mathcal{H}} tr(p(\underline{A})), \\ & s.t\, q_i(\underline{A}) \geq 0. \\ & \quad tr(r_j(\underline{A})) \geq 0 \end{aligned} \tag{3}$$

Moi:Maybe you can elaborate and extend the last definition

Partially commutative polynomial optimization is a polynomial optimization in the partially-commutative algebra [see [**<empty citation>**] for details]. Informally speaking, given a set of variables X and a graph $\mathcal{G} = (X, E)$, whose vertices are letters that are connected if they commute, we denote by $X^{\mathcal{G}}$ the partially-commutative variables and the partially commutative polynomial optimization reads as,

$$\begin{aligned} p^* = \inf_{\underline{A}\in\mathcal{B}(\mathcal{H})^n,\psi\in\mathcal{H},\mathcal{H}} & \langle\psi, \ulcorner p\urcorner(\underline{A})\psi\rangle\,, \\ s.t\ \ulcorner q_i\urcorner(\underline{A}) &\geq 0, \\ s.t\ \langle\psi, \ulcorner r_i\urcorner(\underline{A})\psi\rangle &\geq 0, \\ s.t\ \langle\psi,\psi\rangle &= 1. \end{aligned} \tag{4}$$

where $\ulcorner p\urcorner$ means the polynomial with partially-commutative variables. The tracial version can be defined similarly as,

$$\begin{aligned} p^* = & \inf tr(\ulcorner p\urcorner(X)), \\ & s.t\ \ulcorner q_i\urcorner(X) \geq 0. \\ & \quad tr(\ulcorner r_j\urcorner(X)) \geq 0 \end{aligned} \tag{5}$$

The algorithms for solving polynomial optimization problems, whether commutative or non-commutative, and their tracial or eigenvalue versions, share the same core idea of building a sequence of SDP relaxations of increasing size, whose optimal values converge to the global optimum under certain assumptions. The SDP relaxations are often indexed by a level parameter, with higher levels yielding better approximations of the global optimum but at the cost of increased computational complexity.

1.1 Motivation for PCPOP

Given how intractable it can be to solve the SDPs for higher levels or bigger problem sizes, i.e., with large number of input variables and many polynomial constraints, several solutions to reduce the problem complexity by harnessing the structure of the input problem have been proposed. For e.g in [WML21], they use the sparsity of the input problem to reduce the SDP size at each levels of hierarchy, in [Ros18] , they propose how to use symmetry in the input problem to reduce the problem size and in [Rus23] they propose efficient solution for special type of correlations called synchronous correlations. But so far, there hasn't been much work on how to leverage the equality constraints in the input problem to reduce the complexity of the SDP relaxations.

The main motivation for PCPOP comes from polynomial optimization problems with equality constraints that appear ubiquitously in physics, especially in the field of quantum information. We found mathematical tools that abstracts these ubiquitous scenarios and PCPOP implements the associated mathematical structures and algorithms to efficiently build and solve the SDP relaxations. Below we discuss three classes of problems that motivated us to build PCPOP

For the first class, consider a quantum experiment, where some operators act jointly on quantum systems(say an unitary describing interaction of two quantum systems) and some operators acting locally (say measurement on a quantum system). So operators that share a quantum system don't commute with each other but operators that don't share a quantum system , that is, they act on space-like separated quantum systems commute with each other. The operators could also satisfy other binomial constraints such as being projectors, unitary, dichotomic, pauli constraints or some pairs of operators being orthogonal. These operators can be abstracted as partially-commutative variables. And the polynomial optimization consisting of such operators can be best solved as a partially-commutative polynomial optimization problem[see 4.2 for examples and how they are implemented].

For the second class, consider a polynomial optimization problem which consists of two types of equality constraints. First being, special commutation rules such that the variables can be partitioned into different subsets where all the variables in a subset share the same commutation relations. The other type of constraints consist of polynomials that act on variables belonging to the same subset. We discuss methods in [**<empty citation>**] to leverage such type of equality constraints as efficiently as possible. The abstract mathematical structure used for this is the *Graph product algebra* which extends the partially-commutative algebra. See section 4.3 for examples and how they are implemented.

The third class of problems consists of trace polynomial optimization problems as in eq 5 but with partially commutative variables and the equality constraints requiring the variables to be projectors, unitaries, or dichotomic. This class of problems appears naturally when considering prepare-and-measure scenarios. We discuss methods in [**<empty citation>**] to build efficient SDP relaxations for such class of problems. See Section **??** for examples and how they are implemented.

1.2 Scope of PCPOP

Here we discuss all the classes of polynomial optimization problems that can be solved using PCPOP.

- Non-commutative polynomial optimization problems, as mentioned in eq 2 and illustrated in section 4.1. PCPOP uses `AbstractAlgebra.jl` package to reduce the polynomials using non-commutative Gröbner basis.
- Partially-commutative polynomial optimization problems, as mentioned in 4 and illustrated in section 4.2. Here the special binomial constraints such as projectors, unitaries, dichotomic, orthogonality etc. are taken care implicitly while building the monomials as clique normal forms.
- Graph product polynomial optimization problems. Check section 1.1 for motivation and 4.3 for examples.
- Tracial polynomial optimization problems with partially commuting variables, as mentioned in eq 5 and illustrated in section **??**. Here the special binomial constraints such as projectors, unitaries, dichotomic, orthogonality etc. are taken care implicitly while building the monomials as cyclic graph normal forms. Moi: Maybe here you add something

1.3 How PCPOP compares to other related libraries

There exists excellent packages of polynomial optimization. To start with, Ncpol2sdpa [Wit15] is a python-based library for solving non-commutative polynomial optimization problems. It is popular in the quantum information community as it was the first library to implement the NPA hierarchy and has easy interface to fully express a non-commutative polynomial optimization problem. QuantumNPA [ewo25] is another library for implementing NPA hierarchy. It is Julia-based and has easy interface to express common quantum information scenarios. Another popular Julia-based library is SumOfSquares [Wei+19] which is excellent for fully-commutative polynomial optimization problems and has support for leveraging sparsity and symmetry in the input problem. It has some support for non-commutative polynomial optimization problems.

Compared to Ncpol2sdpa, QuantumNPA and SumOfSquares, PCPOP has certain advantages like support for trace polynomial optimization, leveraging symmetries in the input problem and efficient handling of equality constraints. Compared to Ncpol2sdpa, PCPOP is based in Julia and doesn't assume normalization, which can be hard to tweak in Ncpol2sdpa. Also, Ncpol2sdpa handles general equality constraints using localizing matrices which is not efficient and binomial constraints using replacement rules which isn't effective [see examples7,8,9]. Both PCPOP and QuantumNPA provide built-in functions to define common quantum operators, but it is hard to impose general equality constraints in QuantumNPA. QuantuNPA can handle partially-commutative varibles that satisfy common binomial constraints but it can be hard to express those and not efficient for larger problems. SumOfSquares has limited support for quantum information specific operators or scenarios.

2 Technical background

PCPOP implements the monomials in partially-commutative monoid through the *clique normal forms* and *graph normal forms* as discussed in [**<empty citation>**]. Here we briefly introduce and illustrate these normal forms.

Given a set of variables $X = \{x_1, x_2, \ldots, x_n\}$, and a graph $\mathcal{G} = (X, E)$ with the vertices as the variables and the edge set capturing the commutation relations, i.e. $(x_i, x_j) \in E$ if and only if x_i and x_j don't commute. This graph is called the independence graph. The dependence graph is the complement of the independence graph. To find the *clique-normal form* $w^{\mathcal{C}}$ for a word w, we project the word onto the maximal cliques of the dependence graph. This is illustrated through the example below.

Example 1. *Say* $X = \{a, b, c\}$ *and* $[a, b] = [b, c] = 0$ *are the only commutations. The dependence graph is as shown below.*

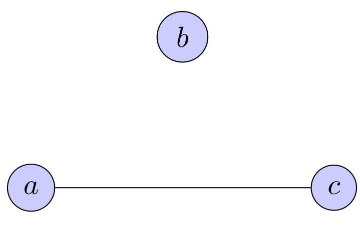

Figure 1: Dependence graph

The maximal cliques are $\{b\}, \{a, c\}$. *Now consider the words* $w_1 = cab$ *and* $w_2 = bca$. *The clique normal forms are* $w_1^{\mathcal{C}} = w_2^{\mathcal{C}} = (ca, b)$.

To compute the graph normal form of a word w, we label the different occurrences of each letter in the word w from left to right. For example, for a word $w = abacb$, we label it as $a_1 b_1 a_2 c_1 b_2$. Then we build the directed graph w^G whose vertices are the labeled letters and there is a directed edge $x \to y$ if the corresponding letters of the labels x, y don't commute and if there is no path between them. It is illustrated as follows.

Example 2. *Say* $X = \{a, b, c\}$ *and* $[a, b] = [b, c] = 0$ *are the only commutations. Consider the word* $w = abacb$*. We label it as* $a_1 b_1 a_2 c_1 b_2$*. The graph* w^G *is as shown below.*

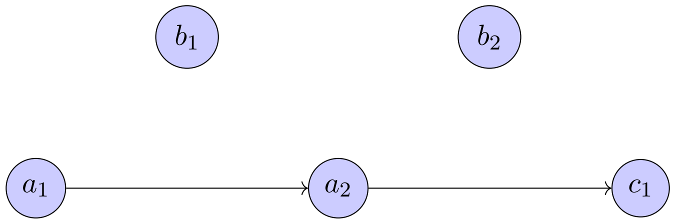

Figure 2: Graph w^G

PCPOP handles special equality constraints like projectors, unitaries, dichotomic, orthogonality etc alongside partial commutations directly in the clique normal form. Although figure 3 illustrates multiplication of graph product monomials, the mechanism to handle special equality constraints in partially commutative monomials is the same.

PCPOP also implements data structures to represent polynomials in graph product algebras and some new algorithms to find normal forms for certain types of graph product algebras and classes of equality constraints [see [**<empty citation>**] for details]. A graph product algebra is the span of monomials belonging to a graph product monoid. Informally speaking, a graph product monoid is a monoid that combines different monoids where some monoids commute and some do not. A partially-commutative monoid can be seen as a special instance of a graph product algebra by replacing the monoids with letters.

PCPOP implements elements of a partially commutative monoid and a graph product monoid using the same data structure called `GraphProductWord{V}`, where V takes the type for variables when it is a partially-commutative monomial and the type for monoid elements when it is a graph product monomial. The data structure `GraphProductWord{V}` essentially consists of three fields, the first being an array of arrays `clique_words` to hold the clique normal form, and the other two are sets (`edge_l,edge_r`) that hold the letters or monoid elements that can be commuted to the edge (left, right edge). Having the fields `edge_l, edge_r` helps in efficiently implementing the multiplication of two graph product words. Below is an example that illustrates how to find normal forms under partial commutations and projectors.

Example 3. *Say we have two quantum systems that belong to Alice and Bob respectively, Alice has two projective measurements* $\{A_0, A_1\}$ *and Bob has two projective measurements* $\{B_0, B_1\}$ *and there are two projectors* $\{AB_0, AB_1\}$ *that act on both Alice's and Bob's systems. We know that Alice's and Bob's measurements commute with each other but not with the joint projectors. The operators belonging to Alice forms the free monoid* $\mathbb{M}_A$*, similarly, operators of Bob form* $\mathbb{M}_B$ *and the joint operators form* $\mathbb{M}_{AB}$*. We combine the monoids* $\{\mathbb{M}_A, \mathbb{M}_B, \mathbb{M}_{AB}\}$ *where* $\mathbb{M}_A, \mathbb{M}_B$ *commute to form the graph product monoid* $\mathbb{M}$*. Informally speaking, the maximal cliques are* $\{\mathbb{M}_A, \mathbb{M}_{AB}\}$ *and* $\{\mathbb{M}_B, \mathbb{M}_{AB}\}$*.*

We can rewrite the words $w_1 = AB_0 B_0 A_1 A_0$ *and* $w_2 = A_1 B_0 AB_1$ *as*

$w_1 = (AB_0)_{\mathbb{M}_{AB}} (B_0)_{\mathbb{M}_B} (A_1 A_0)_{\mathbb{M}_A}$ *and* $w_2 = (A_1)_{\mathbb{M}_A} (B_0)_{\mathbb{M}_B} (AB_1)_{\mathbb{M}_{AB}}$*, where we multiply adjacent letters that belong to the same monoid and represented the result as word in respective monoid.*

Now if we multiply them, we get $w_1 w_2 = (AB_0)_{\mathbb{M}_{AB}} (B_0)_{\mathbb{M}_B} (A_1 A_0 A_1)_{\mathbb{M}_A} (AB_1)_{\mathbb{M}_{AB}}$*, as* B_0 *can commute through* A_1 *and* A_0*. In their clique normal forms, the multiplication algorithm is illustrated below.*

$$
\begin{array}{ccc}
w_1 & & w_2 \\
\begin{bmatrix} (AB_0)(A_1A_0) \\ \\ (AB_0)(B_0) \end{bmatrix} & \times & \begin{bmatrix} (A_1)(AB_1) \\ \\ (B_0)(AB_1) \end{bmatrix} \\
\{AB_0\} \quad \{(B_0),\ (A_1A_0)\} & & \{(A_1),(B_0)\} \quad \{(AB_1)\}
\end{array}
$$

$$=$$

$$
\begin{array}{ccccc}
w'_1 & & m & & w'_2 \\
\begin{bmatrix} (AB_0)(A_1A_0) \\ \\ (AB_0) \end{bmatrix} & \times & \begin{bmatrix} \\ \\ (B_0) \end{bmatrix} & \times & \begin{bmatrix} (A_1)(AB_1) \\ \\ (AB_1) \end{bmatrix} \\
\{AB_0\} \quad \{(A_1A_0)\} & & \{(B_0)\} \quad \{(B_0)\} & & \{(A_1),(AB_1)\} \quad \{(AB_1)\}
\end{array}
$$

$$=$$

$$
\begin{array}{c}
\begin{bmatrix} (AB_0)(A_1A_0A_1)(AB_1) \\ \\ (AB_0)(B_0)(AB_1) \end{bmatrix} \\
w_1 * w_2
\end{array}
$$

Figure 3: Multiplication of graph product monomials (column vector view). Two words in their clique normal forms are multiplied. The clique normal forms are shown as column vectors(within square brackets) where each row has the projection of the word to the respective maximal clique. Letters within a simple bracket belong to same monoid. The set below and left(right) to the clique normal forms represent the monoid elements that can be commuted to the left(right) edge. To multiply w_1, w_2 we check which monoid elements on the right edge of w_1 can be multiplied with the left edge elements of w_2. We represent the result of the multiplication in the middle as m. We update the clique normal forms and the left and right edge sets of w_1, w_2 accordingly to $w'_1 w'_2$. We then multiply $w'_1 * m * w'_2$ and we continue iteratively. PCPOP handles constraints where variables belong to same monoid directly in multiplication and rest using Macaulay matrix.

3 Installation

4 Getting started with PCPOP

This section has six parts: (1–4) solving four classes of polynomial optimization—non-commutative, partially-commutative, graph-product, and trace partially-commutative; (5) leveraging symmetries; and (6) defining complex hierarchy levels via a simple interface. Throughout, we use the same notation for a polynomial and its evaluation on a tuple of quantum operators.

4.1 Non-commutative polynomial optimization

Example 4. *Consider the following problem with Hermitian variables X_1, X_2 from [PNA10] and already solved in Ncpol2sdpa[Wit15].*

$$
\begin{aligned}
p^* = \underset{\underline{X},\psi,\mathcal{H}}{inf} \quad & \langle \psi, X_1X_2 + X_2X_1\psi \rangle, \\
s.t \qquad & X_1^2 - X_1 = 0, \\
& -X_2^2 + X_2 + 1/2 \geq 0, \\
& \langle \psi, \psi \rangle = 1.
\end{aligned}
\tag{6}
$$

The Julia code to solve the above problem using PCPOP is given below.

```
# To define the non-commutative variables X1,X2 and its monoid M
@ncmonoid M X1 X2
#  To define the equality constraints
Projector(X1)
build(M)

# To define the objective function, inequality constraints and level of
relaxation
obj=X1*X2+X2*X1
op_ge=[X2-X2^2+0.5]
level=2

# To solve the non-commutative polynomial optimization problem
ov,model,var_dict,principal_matrix=npa(obj,level;op_ge=op_ge)

println("Optimal value is ",ov)
```

To solve a non-commutative polynomial optimization problem using PCPOP, we follow the following steps

- First we need to define the non-commutative variables and an object that stores information about the corresponding monoid using the macro `@ncmonoid`. The first argument is always for the monoid object and the rest for the variables.
- Then we define the equality constraints using `add_relations!`. For constraints like *Projector, Unitary, Unipotent* we can use built-in functions `Projector,Unitary,Unipotent` respectively.
- Finally, we call the function `npa` to solve the optimization problem. `npa` takes the objective polynomial and the level as compulsory arguments and also some optional arguments like arrays of operator inequalities (`op_ge`), operator equalities (`op_eq`), expectation value equalities (`tr_eq`) and inequalities (`tr_ge`). The function returns the optimal value, the SDP model, a dictionary that maps the monomial basis to corresponding JuMP variables and finally the principal matrix.

PCPOP provides a nice interface to declare variables. We can create a container of variables using `X[n,m]` where X is the name for the variable array, n is the number of Hermitian variables and m is the number of non-Hermitian variables. In that case X stores the Hermitian variables and $X\_$ stores the non-Hermitian variables.

```
julia> @ncmonoid M X[2,1] a_
julia> X
2-element Vector{cgb.Variable}:
X_1
X_2
julia> X_
1-element Vector{cgb.Variable}:
X_1_
julia>X_[1]'
X_1__
julia> cgb.is_herm(a_)
false
julia> a_'
a__
```

We can add relations using the built-in function add_relations! which takes as input an array of non-commutative polynomials. When the monoid is built using build(M), the AbstractAlgebra package is used to compute the Gröbner basis for the ideal generated by the relations and then it is used to reduce the polynomials. It is illustrated below.

```
julia>@ncmonoid M a b
julia>cgb.add_relations!(M,[a^2 - a - 1, a * b - b * a])
julia>cgb.build(M)
julia>a*b*a*a
2.0b * a + 1.0b
```

4.2 Partially-commutative polynomial optimization

Example 5. *Consider the problem of certifying genuine multipartite nonlocality in a Bell scenario with n parties where each party performs two dichotomic measurements [AMR24]. Here we illustrate the case for two parties. This can be cast as a polynomial optimization problem with variables $X = \{A_1, A_2, B_1, B_2\}$, each being a projector, and where A_i, B_j belong to Alice and Bob, respectively. The problem reads as follows.*

$$
\begin{aligned}
p^* = \ & \underset{\underline{X},\psi,\mathcal{H}}{inf} \ \langle \psi, A_1 B_1 \psi \rangle, \\
s.t \quad & [A_i, B_j] = 0, \forall i, j \in \{1,2\} \\
s.t \quad & A_i^2 - A_i = B_j^2 - B_j = 0, \forall i, j \in \{1,2\} \\
s.t \quad & \langle \psi, A_2 B_1 \psi \rangle = \langle \psi, A_1 B_2 \psi \rangle = 0, \\
s.t \quad & \langle \psi, (1 - A_2)(1 - B_2) \psi \rangle = 0, \\
& \langle \psi, \psi \rangle = 1.
\end{aligned} \tag{7}
$$

This can be solved with PCPOP as follows.

```
@pcmonoid M A[2,0] B[2,0]
Projector.([A;B])
@comms A B
build(M)
obj=A[1]*B[1]
tr_eq= [[A[2] * B[1],0],[A[1] * B[2],0],[(1-A[2]) * (1-B[2]),0]]
level=2
ov,model,var_dict,principal_matrix=npa(obj,level;tr_eq=tr_eq,min=false)
println("Optimal value is ",ov)
```

This returns the optimal value 0.0902 as in [AMR24]. Instances of the previous example for $n = 2, 3, 4$ parties and for `level` $= 2, 3, 4$ *have been solved and benchmarked in Section 5.*

The steps to solve a partially-commutative polynomial optimization problem are similar to that of non-commutative polynomial optimization with some differences.

- First we define the partially-commutative monoid using the macro `@pcmonoid`. The first argument is always for the monoid object and the rest for the variables.
- Then, we define which variables commute using the built-in macro `@comms`.
- Then we define some special operator equality constraints like *projector*, *unitary*, *dichotomic*, and *orthogonal pairs* using built-in functions `Projector`,`Unitary`,`Unipotent` and the macro `@ortho`, respectively. We leave the general equality constraints to be fed to the `npa` function as the optional argument `op_eq`.

- Finally, we call the function `npa` to solve the optimization problem. `npa` takes the objective polynomial and the level as compulsory arguments and also some optional arguments like arrays of operator inequalities (`op_ge`), operator equalities (`op_eq`), expectation-value equalities (`tr_eq`) and inequalities (`tr_ge`). The function returns the optimal value, the SDP model, a dictionary that maps the monomial basis to corresponding JuMP variables, and the principal matrix.

Remark 6. *Notice that compared to the non-commutative case, where we enforce all equality constraints before building the monoid, here we only enforce the special equality constraints like projector, unitary, dichotomic, and orthogonality before building the monoid. These special equality constraints are handled directly and efficiently using normal forms and not using the Macaulay method, which handles the general operator equality constraints.*

The user interface to define the variables and the monoid is same as in non-commutative case. The macro `@comms` provides multiple ways to define commutation relations. We illustrate all the options.

```
julia>@pcmonoid M a b # a and b are two hermitian variables
julia>@comms a b #  variable a commutes with variable b
julia>print(a*b,"\n",b*a)
(a)(b)
(a)(b)
julia> a*b==b*a
true
```

```
julia> @pcmonoid M A[2,0] B[2,0] # A1,A2 are hermitian variables for Alice and
     B1,B2 for Bob
julia> @comms A B # all variables of A commute with all variables of B
```

```
julia>@pcmonoid M a b c d e f # a,b,c,d,e,f are six hermitian variables
julia>@comms [a,b,c] # to enforce [a,b]=[b,c]=[a,c]=0
julia>a*b*c==c*a*b
true
julia>a*e*b == b*e*a
false
```

To enforce the orthogonality constraint $a * b = 0$, we can use the built-in macro `@ortho` as `@ortho a b`. The `@ortho` macro works similarly as `@comms` macro with the exception that it doesn't assume $b * a = 0$ when $a * b = 0$.

For certain equality constraints, the full Gröbner basis is infinite; even reductions using finite truncations (at a fixed degree) remain costly, and the substitution method in the Ncpol2sdpa package [Wit15] is ineffective in that case. We illustrate this with examples of partial commutations whose independence graph have non-transitive orientation[see [**<empty citation>**] for details] and in which some variables are projectors.

Example 7. *Consider* $X = \{a, b, c\}$ *where* a, b *are projectors and* $[a, b] = [b, c] = 0$*. The independence graph is as shown below.*

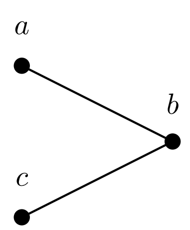

Figure 4: Independence graph

We consider the following optimization problem.

$$\begin{aligned} p^* = & \inf_{\underline{X},\psi,\mathcal{H}} \langle\psi| cab - bca |\psi\rangle, \\ s.t \quad & [a,b] = [b,c] = 0 \\ s.t \quad & a^2 - a = b^2 - b = 0 \\ & \langle\psi|\psi\rangle = 1. \end{aligned} \tag{8}$$

With PCPOP, the above problem can be solved as follows.

```
@pcmonoid M a b c
Projector.([a,b])
add_comms!([(a,b),(b,c)])
build(M)
obj=c*a*b - b*c*a
level=2
ov,model,var_dict,principal_matrix=npa(obj,level)
println("Optimal value is ",ov)
```

Example 8. *Consider* $X = \{a,b,c,d\}$ *where* a,b *are projectors and* $[a,b] = [b,c] = [c,d] = [a,d] = 0$*. The independence graph is as shown below.*

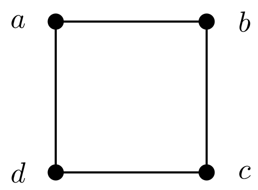

Figure 5: Independence graph

We consider the following optimization problem.

$$\begin{aligned} p^* = & \inf_{\underline{X},\psi,\mathcal{H}} \langle\psi| cd^2b - d^2bc |\psi\rangle, \\ s.t \quad & [a,b] = [b,c] = [c,d] = [a,d] = 0 \\ s.t \quad & a^2 - a = b^2 - b = 0 \\ & \langle\psi|\psi\rangle = 1. \end{aligned} \tag{9}$$

With PCPOP, the above problem can be solved as follows.

```
@pcmonoid M a b c d
Projector.([a,b])
add_comms!([(a,b),(b,c),(c,d),(a,d)])
build(M)
obj=c*d^2*b - d^2*b*c
level=2
ov,model,var_dict,principal_matrix=npa(obj,level)
println("Optimal value is ",ov)
```

Example 9. *Consider* $X = \{a,b,c,d,e\}$ *where* a,b *are projectors and* $[a,b] = [b,c] = [c,d] = [d,e] = [a,e] = 0$*. The independence graph is as shown below.*

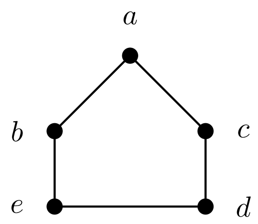

Figure 6: Independence graph

We consider the following optimization problem.

$$
\begin{aligned}
p^* = & \inf_{\underline{X},\psi,\mathcal{H}} \langle\psi| de^2c - e^2dc |\psi\rangle, \\
s.t \quad & [a,b] = [b,c] = [c,d] = [d,e] = [a,e] = 0 \\
s.t \quad & a^2 - a = b^2 - b = 0 \\
& \langle\psi|\psi\rangle = 1.
\end{aligned} \tag{10}
$$

With PCPOP, the above problem can be solved as follows.

```
@pcmonoid M a b c d e
Projector.([a,b])
add_comms!([(a,b),(b,c),(c,d),(d,e),(a,e)])
build(M)
obj=d*e^2*c - e^2*d*c
level=2
ov,model,var_dict,principal_matrix=npa(obj,level)
println("Optimal value is ",ov)
```

4.3 Graph product polynomial optimization

Consider three groups(A,B,AB) where each group has two variables and so, $X = \{X_{j,i} : i \in \{A,B,AB\}, j \in \{1,2\}\}$. All the variables of group A commute with all the variables of group B but not with the variables of group AB. The variables X_1^i, X_2^i of each group i follow the constraints as in problem 6. The optimization problem we consider reads as follows.

$$
\begin{aligned}
p^* = & \inf_{\underline{X},\psi,\mathcal{H}} \langle\psi| \Big[\sum_{i\in\{A,B,AB\}} X_{1,i}X_{2,i} + X_{2,i}X_{1,i} + \sum_{i,j\in\{A,B,AB\}} X_{1,i} * X_{2,j} \Big] |\psi\rangle, \\
s.t \quad & X_{1,i}^2 - X_{1,i} = 0, \forall i \in \{A,B,AB\} \\
& [X_{i,A}, X_{j,B}] = 0, \forall i,j \in \{1,2\} \\
& -X_{2,i}^2 + X_{2,i} + 1/2 \geq 0, \forall i \in \{A,B,AB\} \\
& \langle\psi,\psi\rangle = 1.
\end{aligned} \tag{11}
$$

With PCPOP, the above problem can be solved as follows.

```
# Monoid fro group A
@ncmonoid A X1A X2A
# Monoid for group B
@ncmonoid B X1B X2B
# Monoid for group AB
@ncmonoid AB X1AB X2AB
vars=[X1A,X2A,X1B,X2B,X1AB,X2AB]
 #All variables are hermitian and projectors. This constraint only involve
 variables of same group.
```

```
Projector.(vars)
# We build each child monoids.
# We create the parent monoid using the child monoids.
M=graph_product_monoid("M",[A,B,AB])
# We enforce the commutation relations between the child monoids, this is
enough to enforce all the required commutation relations.
@comms A B
# Build the parent monoid, it automatically builds the child monoids.
build(M)
# Define the objective function.
obj_local=X1A*X2A + X2A*X1A + X1B*X2B + X2B*X1B + X1AB*X2AB + X2AB*X1AB
obj_cross=X1A*X2B + X1A*X2AB + X1B*X2A + X1B*X2AB + X1AB*X2A + X1AB*X2B
obj= obj_local + obj_cross
# Define operator inequalities.
op_ge=[i-i^2+0.5 for i in vars]
# Define  level of relaxation.
level=2
# Solve the graph product polynomial optimization problem.
ov,model,var_dict,principal_matrix=npa(obj,level;op_ge=op_ge)
println("Optimal value is ",ov)
```

The steps to solve a graph product polynomial optimization problem are as follows.

- First we define the child monoids. One can use already-defined macros like `@ncmonoid` , `@pcmonoid` or create custom monoids.
- Then we enforce operator equality constraints on the child monoids and build them.
- Then we create the parent monoid using the built-in function `graph_product_monoid` which takes as input, the name of the parent monoid and an array of child monoids.
- Then we define the commutation relations between the child monoids using the built-in macro `@comms`.
- Then we build the parent monoid using `build(M)`.
- Finally, we call the function `npa` to solve the optimization problem. You can give operator equalities that act on variables from different child monoids using the optional argument `op_eq` of `npa` function. As before, you can also give operator inequalities(`op_ge`), expectation value equalities(`tr_eq`) and inequalities(`tr_ge`) using the respective optional arguments of `npa` function.

One can simplify the above code using the built-in macro `@subsystem` as follows.

```
# Directly create parent monoid M with child monoids A,B,AB each having
two hermitian variables.
@subsystem M A[2,0] B[2,0] AB[2,0]
# All the variables of parent monoid can be accessed using variables(M).
The variables are {a1,a2,b1,b2,ab1,ab2}.
vars=variables(M)
# Enforcing projector constraint on all the variables. These constraints
are handled using Grobner basis reduction.
Projector.(vars)
# Building parent parent monoid, automatically build child monoids.
build(M)
# Enforcing operator inequalities on all the variables.
op_ge=[i-i^2+0.5 for i in vars]
# Define objective function.
```

```
obj_local = a[1]*a[2] + a[2]*a[1] + b[1]*b[2] + b[2]*b[1] + ab[1]*ab[2] +
ab[2]*ab[1]
obj_cross = a[1]*b[2] + a[1]*ab[2] + b[1]*a[2] + b[1]*ab[2] + ab[1]*a[2] +
 ab[1]*b[2]
obj = obj_local + obj_cross
# Define level of relaxation.
level=2
# Solve the graph product polynomial optimization problem.
ov,model,var_dict,principal_matrix=npa(obj,level;op_ge=op_ge)
println("Optimal value is ",ov)
```

4.4 Trace polynomial optimization

Consider the problem, studied in [Tav+22b], of characterizing correlations in a prepare-and-measure scenario where the information capacity of the ensemble of prepared quantum states is constrained. The trace polynomial optimization problem reads as follows. With PCPOP, we reproduce Figure 3 in [Tav+22b].

```
G=0.8  #For other values of G, change G and rerun the code.
@pcmonoid M ρ[3,0] σ A[2,0]
Projector.(A)
build(M)
PA=[A[1] A[2];1-A[1] 1-A[2]]
# The following two lines impose information constraint.
ge = [σ-1/3*ρ[i] for i in 1:3]
ge_constr = [[-σ,-G]]
# To impose the constraint that the states are mixed states.
ge=vcat(ge,[i-i*i for i in ρ])
# Normalization of states.
eq_constr = [[ρ[x],1] for x in 1:3]
wit = -(2*PA[1,1]-1)*ρ[1]-(2*PA[1,2]-1)*ρ[1]-(2*PA[1,1]-1)*ρ[2]+(2*PA
[1,2]-1)*ρ[2]+(2*PA[1,1]-1)*ρ[3]
level = 2 # this is the level of the localising matrices and it will be
enough to retrieve the values of the plot (they use level 3 of the
principal moment matrix)
ov,_,_=npa(wit,level;op_ge=ge,tr_eq=eq_constr,tr_ge=ge_constr,cyclic=true,
min=false,normalize=false)
# Notice here we use cyclic=true to do tracial reductions.
```

Moi: Maybe you can extend with more examples that solves general state polynomial optimization problems.

4.5 Handling symmetries in partially-commutative polynomial optimization

In many optimization problems, the symmetries in the problem can be leveraged to simplify the problem. Moi, maybe you can expand this section with an example and how to use it in PCPOP.

4.6 Handling intermediate levels of hierarchy

PCPOP provides excellent and simple interface to define intermediate levels of hierarchy. This is useful when some level is not enough and the next integer level is too large to solve. We illustrate all the available ways to define intermediate levels.

Consider three groups X_1, X_2, X_3 where each group could be an array of variables whose names start with the name of the respective group or it could be monoids with names being X_1, X_2, X_3 respectively.

```
# Define three groups each having two hermitian variables.
julia>@pcmonoid M X1[2,0] X2[2,0] X3[2,0]
# This defines level where the index monomials are the union of three groups.
#First, all the level 2 monomials. By level 1 we mean the union of all
    variables of X1,X2,X3 and the identity monomial and by level 2, all the
    two-products of level 1 monomials.
#Second, all the monomials that are product of one variable in array X1 and
    one in array X2.
#Third, all the monomials that are product of one variable in array X2 and one
     in array X3.
julia>level="2+X1*X2+X2*X3"
#Here we define three non-commuative monoids each having two hermitian
    variables and where monoid A commutes with monoid B.
julia>@subsystem M A[2,0] B[2,0] AB[2,0]
# This defines level where the index monomials are the union of three groups.
#First, all the level 2 monomials.
#Second, all the monomials that are product of one variable in monoid X1 and
    one in monoid X2.
#Third, all the monomials that are product of one variable in monoid X2 and
    one in monoid X3.
julia>level="2+X1*X2+X2*X3"
julia> @pcmonoid M X1[3,0] X2[5,0]
# All monomials that are product of first two variables in X1 and first three
    variables in X2.
julia> level="X1[2]*X2[3]"
```

5 Benchmarking PCPOP

We report the performance of PCPOP on examples considered before. We use the size of the principal moment matrix, number of scalar variables and cpu time as measure of performance on a problem at specific level of relaxation.

We start with the example 5 as it can be generalized to any number of parties to show genuine multipartite entanglement. We consider

Table 1: Benchmark summary for PCPOP relaxations. SDP size refers to the principal moment matrix dimension; CPU time measured in seconds.

# of parties	level	SDP size	# scalar vars	CPU time (s)
2	2	13	31	0.127
2	3	25	61	0.612
2	4	41	101	1.917
3	2	25	93	0.516
3	3	63	250	5.189
3	4	129	529	45.915
4	2	41	229	1.646
4	3	129	833	44.58
4	4	321	2241	643.277

Now, we compare the performance of PCPOP with Ncpol2sdpa on the examples $7, 8, 9$. We compare the size of the principal moment matrix and the number of scalar variables for both packages at the same relaxation order.

Table 2: Comparison of PCPOP and Ncpol2sdpa by relaxation order. SDP size refers to the principal moment matrix dimension.

Example	level	Gröbner size	Package	SDP size	# scalar vars
7	2	6	PCPOP	9	23
			Ncpol2sdpa	9	25
7	3	8	PCPOP	17	58
			Ncpol2sdpa	18	73
7	4	10	PCPOP	30	142
			Ncpol2sdpa	35	223
8	2	10	PCPOP	15	60
			Ncpol2sdpa	15	64
8	3	14	PCPOP	37	219
			Ncpol2sdpa	40	284
8	4	18	PCPOP	82	723
			Ncpol2sdpa	96	1339
9	2	13	PCPOP	24	176
			Ncpol2sdpa	24	184
9	3	19	PCPOP	85	1686
			Ncpol2sdpa	89	1934
9	4	25	PCPOP	287	17226
			Ncpol2sdpa	315	22126

On top of improvements in the size of the principal moment matrix and number of scalar variables, PCPOP also provides tighter bounds as compared to Ncpol2sdpa. This is obvious as PCPOP imposes more constraints on th emoment matrix.

6 Conclusion

PCPOP is the latest contribution to the community of polynomial optimization in Julia. It emphasizes leveraging equality constraints to reduce the complexity of SDP relaxations. It has specialized algorithms to handle constraints that appear ubiquitously in quantum information. It has a robust and easy-to-use user interface that can handle a large variety of problem inputs. At last, it provides support for symmetry reduction and trace polynomial optimization, which is missing in most other polynomial optimization packages.

7 Acknowledgements

The authors would like to thank Prof. Stefano Pironio for his valuable feedback and suggestions to improve the package.

References

[AMR24] Ranendu Adhikary, Abhishek Mishra, and Ramij Rahaman. "Self-testing of genuine multipartite entangled states without network assistance". In: *Phys. Rev. A* 110 (1 July 2024), p. L010401. DOI: 10.1103/PhysRevA.110.L010401. URL: https://link.aps.org/doi/10.1103/PhysRevA.110.L010401.

[Bur+13] Sabine Burgdorf et al. "The tracial moment problem and trace-optimization of polynomials". In: *Mathematical Programming* 137.1 (Feb. 2013), pp. 557–578. ISSN: 1436-4646. DOI: 10.1007/s10107-011-0505-8. URL: https://doi.org/10.1007/s10107-011-0505-8.

[ewo25] ewoodhead. *QuantumNPA.jl*. https://github.com/ewoodhead/QuantumNPA.jl. commit 9ec9eea4b5a864874d52c23da96cf639395b9734. 2025.

[Las01] Jean B. Lasserre. "Global Optimization with Polynomials and the Problem of Moments". In: *SIAM Journal on Optimization* 11.3 (2001), pp. 796–817. DOI: 10.1137/S1052623400366802. eprint: https://doi.org/10.1137/S1052623400366802. URL: https://doi.org/10.1137/S1052623400366802.

[NPA08] Miguel Navascués, Stefano Pironio, and Antonio Acín. "A convergent hierarchy of semidefinite programs characterizing the set of quantum correlations". In: *New Journal of Physics* 10.7 (July 2008), p. 073013. DOI: 10.1088/1367-2630/10/7/073013. URL: https://doi.org/10.1088/1367-2630/10/7/073013.

[PNA10] S. Pironio, M. Navascués, and A. Acín. "Convergent Relaxations of Polynomial Optimization Problems with Noncommuting Variables". In: *SIAM Journal on Optimization* 20.5 (2010), pp. 2157–2180. DOI: 10.1137/090760155. eprint: https://doi.org/10.1137/090760155. URL: https://doi.org/10.1137/090760155.

[Pri+23] Ignatius W. Primaatmaja et al. "Security of device-independent quantum key distribution protocols: a review". In: *Quantum* 7 (Mar. 2023), p. 932. ISSN: 2521-327X. DOI: 10.22331/q-2023-03-02-932. URL: https://doi.org/10.22331/q-2023-03-02-932.

[Ros18] Denis Rosset. *SymDPoly: symmetry-adapted moment relaxations for noncommutative polynomial optimization*. 2018. arXiv: 1808.09598 [quant-ph]. URL: https://arxiv.org/abs/1808.09598.

[Rus23] Travis B. Russell. *A synchronous NPA hierarchy with applications*. 2023. arXiv: 2105.01555 [quant-ph]. URL: https://arxiv.org/abs/2105.01555.

[Tav+22a] Armin Tavakoli et al. "Bell nonlocality in networks". In: *Reports on Progress in Physics* 85.5 (Mar. 2022), p. 056001. DOI: 10.1088/1361-6633/ac41bb. URL: https://doi.org/10.1088/1361-6633/ac41bb.

[Tav+22b] Armin Tavakoli et al. "Informationally restricted correlations: a general framework for classical and quantum systems". In: *Quantum* 6 (Jan. 2022), p. 620. ISSN: 2521-327X. DOI: 10.22331/q-2022-01-05-620. URL: https://doi.org/10.22331/q-2022-01-05-620.

[Tav+24] Armin Tavakoli et al. "Semidefinite programming relaxations for quantum correlations". In: *Rev. Mod. Phys.* 96 (4 Dec. 2024), p. 045006. DOI: 10.1103/RevModPhys.96.045006. URL: https://link.aps.org/doi/10.1103/RevModPhys.96.045006.

[Wan+24] Jie Wang et al. "Certifying Ground-State Properties of Many-Body Systems". In: *Phys. Rev. X* 14 (3 July 2024), p. 031006. DOI: 10.1103/PhysRevX.14.031006. URL: https://link.aps.org/doi/10.1103/PhysRevX.14.031006.

[Wei+19] Tillmann Weisser et al. "Polynomial and Moment Optimization in Julia and JuMP". In: *JuliaCon*. 2019. URL: https://pretalx.com/juliacon2019/talk/QZBKAU/.

[Wit15] Peter Wittek. "Algorithm 950: Ncpol2sdpa—Sparse Semidefinite Programming Relaxations for Polynomial Optimization Problems of Noncommuting Variables". In: *ACM Trans. Math. Softw.* 41.3 (June 2015). ISSN: 0098-3500. DOI: 10.1145/2699464. URL: https://doi.org/10.1145/2699464.

[WML21] Jie Wang, Victor Magron, and Jean-Bernard Lasserre. "TSSOS: A Moment-SOS Hierarchy That Exploits Term Sparsity". In: *SIAM Journal on Optimization* 31.1 (2021), pp. 30–58. DOI: 10.1137/19M1307871. eprint: https://doi.org/10.1137/19M1307871. URL: https://doi.org/10.1137/19M1307871.